

MLib C++ Coding Standards

Nicholas Burgess



MHI

Nicholas.Burgess@uk.mizuho-sc.com

22nd March 2016

MLib Analytics

- MLib is Mizuho's C++ Pricing & Analytics Library
- Developed by MHSC Tokyo & MHI London

Coding Standards

- Facilitate Accurate Pricing & Risk
- Protect the Bank from Adverse Code Changes & Pricing Errors
- Reduce Development Time & Cost
- Encourage Teamwork, Collaboration & Build Economies of Scale

1 Minimum Requirements & Breaches

- Code Reviews
- Existing Tests Must Pass
- New Functionality & New Tests
- Backwards Compatibility

2 Guidelines

- Policy & Organization
- Design Style
- Coding Style

3 Examples

- Naming Conventions
- Coding Style & Flow
- Language Features

4 Recommended Reading

Minimum Requirements:

- ① Code Must be Peer Reviewed **BEFORE** Committing to Library
- ② All Existing Code-, Pricing- & Risk-Tests Must Pass
- ③ New Functionality Must be Demo'd & New Tests Put in Place
- ④ Backwards Compatibility Must be Preserved

ACTION POINT: Dealing with Minimum Requirement Breaches

Offending code must be reverted and removed from the library immediately. It is vital that the bank's pricing & risk analytics are protected from any potentially harmful & unvetted changes.

Code Reviews

- Code must be peer reviewed **BEFORE** committing to the library
- Must be on a line-by-line basis
- Should include project property settings
- Should include any unit or regression tests
- New functionality should be demonstrated as part of the review, in Excel perhaps

Google Tests

Google Test is employed as both a unit and regression testing framework and supports test recording and playback. Any new functionality or code change must not break existing tests

Test Rules

- New code changes must not break code tests
- Test breaks must not be hidden or manipulated to force through a code change
- It is the responsibility of the test breaker to fix all broken tests

New Functionality & Tests

- New Functionality Must be Demo'd & Tested, Perhaps in Excel
- New Tests Should Be Added to the Google Test Framework to Protect New Code from Adverse Changes
- Sufficient Test Coverage Must Be Added

Preserve Backwards Compatibility

- Using Optional Arguments & Features
- Optional Features Should Default to the Original Code Behavior
- Writing Alternative Functions & Methods

Protecting the Bank, Downstream Pricing Systems & Users

It is **Critical** that new code should not break end-user, system and downstream pricing & risk systems. Where backward compatibility is not possible a new function or method should be written.

Exceptions

- Documentation, Excel Example Templates & Pricer Check-Ins Do Not Require Code Reviews
- Bug Fixing May Break Tests but Function Signatures, Methods & Parameters Must Remain Backward Compatible
- It is the Responsibility of the Code Breaker to Fix All Broken Tests
- Backward Compatibility Does Not Apply to UAT Versions of the Analytics, Only Production. This is to Allow for UAT Functionality to Receive Feedback & Be Improved.

Policy & Organization

- Compile Cleanly at High Warning Levels
- Use an Automated Build System
- Use a Version Control System
- Invest in Code Reviews

Design Style

- ① Correctness, Simplicity and Clarity Come First
- ② Add Comment Blocks to Classes & Functions
- ③ Code for Scalability
- ④ Don't Optimize Prematurely
- ⑤ Don't Pessimize Prematurely
- ⑥ Ensure Resources are Owned & Use Smart-Pointers

Coding Style

- ① Prefer compile- & link-time errors to run-time errors
- ② Avoid Magic Numbers
- ③ Avoid Abbreviations
- ④ Avoid Raw Pointers, Use Smart-Pointers
- ⑤ Always initialize Variables
- ⑥ Avoid Long Functions & Deep Nesting
- ⑦ Pass Parameters by Value & Reference Appropriately
- ⑧ Always Write Internal `#include` Guards
- ⑨ Don't write using namespace before an include
- ⑩ Actively & Appropriately use the `const` keyword

Function Signatures: Ease of Navigation

Function signatures should be carefully chosen, grouped by prefix for ease of navigation and abbreviations should be avoided e.g.

- mirSwapParRate
- mirSwapNPV
- mirSwapPV01
- mirSwapDV01

Class Member Functions: Expressing Action

Class member functions should clearly indicate the action they perform and abbreviations should be avoided e.g.

- setSwapSchedule
- getSwapSchedule
- calculateSwapAnnuityTerm
- calculateSwapFloatLegPV

1. Naming Conventions

General Naming Rules

- Names should be as descriptive as possible, within reason.
- Do not worry about saving horizontal space as it is far more important to make your code immediately understandable by a new reader.
- Do not use abbreviations that are ambiguous or unfamiliar to readers outside your project;
- Do not abbreviate by deleting letters within a word.
- Use upper case letters as word separators and lower case for the rest of a word.
- Underscore ('_') is only allowed as word separator when the words that constitute a name are themselves standard abbreviations and acronyms.

```
int priceCountReader;    // No abbreviations.  
  
int numErrors;           // "num" is a widespread convention.  
  
int num_DNS_Connections; // Most people know what "DNS" stands for.
```

```
int n;                   // Meaningless.  
  
int nerr;                // Ambiguous abbreviation.  
  
int wgcConnections;     // Only your group knows what this stands  
for.  
  
int pcReader;           // Lots of things can be abbreviated "pc".  
  
int cstmrId;            // Deletes internal letters
```

Naming Rules for Types, Objects and Methods

- Must conform to the General Naming Rules
- Names representing types must start with upper case
- Variable names must start with lower case
- Names representing methods or functions must start with lower case

```
class MyCollections;           // Type name starts with upper case  
  
MyCollections collectionA;     // variable name starts with lower case  
  
void getName();               // method name starts with lower case
```

Other naming rules:

- Global variables should always be referred to using the scope operator, ::
- Class data member names must be prefixed with ''
- Constants and enumeration values must be all in upper case using underscore to separate words

2. General Coding Style and Flow Control

- Every file that contains source code must be documented with an introductory comment that provides information on the file name and its contents.

All files must include copyright information and all comments must be written in English.

```
//  
// @File: test.cc  
// @Description: This is a test program  
// @Created: 16 Feb 2016  
// @Author: Joseph Ye  
// @Department: ISO Front Office Development  
//  
// The copyright to the computer program(s) herein  
// is the property of Mizuho International.
```



- Every *header file* must have a mechanism in place that prevents multiple inclusions of the file in a single compilation.

There are two standard approaches to achieve this:

1. Use `#define` to set an include guard macro, the name of which is picked by the programmer to be unique to that file:

```
#ifndef GRANDPARENT_H
#define GRANDPARENT_H
    ... contents of grandparent.h
#endif /* !GRANDPARENT_H */
```

2. Using pre-processor directive *`#pragma once`*

```
#pragma once

struct foo
{
    int member;
};
```

The second approach is recommended for simplicity and avoidance of name clashes.

- Every class and function declaration and definition must be provided with a description

```
// @Description: This function calculates the area of a square
// @param x [in]  Length of square
// @param y [in]  Width of square
// @return  the area of the square
double getSquareArea(double length, double width);

// @Description: This class is the base class of all swap classes
class baseSwap
{
// ...
};
```

- Place brace under and inline with keywords. Do not place the initial brace on the same line as the keyword

```
// Do not place the initial brace on the same initial line

if (condition) {      while (condition) {
    ...                ...
}                      }
```

- All if, while and do statements require braces even if there is only a single statement within the braces

```
if (1 == somevalue)
{
    somevalue = 2;
}
```

- There should be only one statement per line unless the statements are very closely related
- Each header file should contain one class (or embedded or very tightly coupled classes) declaration only.

This makes it easier to read the source code files. This also improves the version control of the files; for example the file containing a stable class declaration can be committed and not changed anymore.

Similarly, implementation files should hold the member function definitions for a single class (or embedded or very tightly coupled classes) as defined in the corresponding header file

- All *switch* statements should have a *default* clause. This is useful because if an additional *enum* value is added, the *switch* statement won't just silently ignore the new value. Instead it should in some way (such as throwing an exception) to notify the programmer that the *switch* statement must be updated.

```
switch(semaphore)
{
case GREEN:
// statement
// break;
case RED:
// statement
// break;
default:
// unforeseen color; it is a bug
// do some action to signal it
}
```

- Only loop control statements should be included in the `for()` construction

```
sum = 0;
for (i = 0; i < 100; i++)
{
    sum += value[i];
}
```

// NOT: for (i = 0, sum = 0; i < 100; i++)
// {
// sum += value[i];
// }

- Try to avoid forward declaration where possible. Just `#include` the headers you need.

There are well known advantages of using forward declarations such as saving compilation time and saving on unnecessary recompilations. However the following disadvantages are considered riskier factors and outweigh the benefits:

1. Forward declarations can hide a dependency, allowing user code to skip necessary recompilation when headers change.
2. A forward declaration may be broken by subsequent changes to the library. Forward declarations of functions and templates can prevent the header owners from making otherwise-compatible changes to their APIs, such as widening a parameter type, adding a template parameter with a default value, or migrating to a new namespace.
3. Certain cases simply won't work with forward declaration such as when the forward declared symbol is being used as a base class.

3. Language Features

Classes

- The public, protected and private sections of a class should be declared in this order.

Within each section, nested types (e.g. enum or class) should appear at the top.

```
class Path {  
    public:  
        Path();  
        ~Path();  
    protected:  
        void draw();  
    private:  
        class Internal  
        {  
            // Path::Internal declarations go here ...  
        };  
};
```

- Do not have any method bodies inside the class definitions in header files. The class definition will be more compact and comprehensible if no implementation can be seen in the class interface.

This rule also applies to inline functions. You can either put them in a separate file, or at the end of the header file, below the class definition.

```
// .h file

class X
{
    public:
        // Even one liner definition is NOT recommended in class
        declaration
        bool insideClass() const { return false; }
        bool outsideClass() const;
};

// Recommended: function definition outside class
inline bool X::outsideClass() const
{
    return true;
}
```

- In child classes, always re-declare virtual functions as virtual. The compiler will know it is virtual, but the human reader may not without having to check on the base classes. This rule of course applies to virtual destructor.

- Never specify *public* or *protected member data* in a class. Otherwise use *struct* instead.
- Always declare a member function *const* unless it changes the object's observable state. This gives a more precise statement of design intent, better readability, more errors caught by the compiler, and sometimes more optimization opportunities.

```
class Point
{
    int x, y;
public:
    int getx() { return x; }    // BAD, should be const as it doesn't
    modify the object's state
    // ...
};

void f(const Point& pt)
{
    int x = pt.getx();         // ERROR, doesn't compile because getx
    was not marked const
}
```

- Use member initializer list rather than initialize member data within constructor, unless you need to do error checks.

Class member variables are automatically constructed using their default constructor prior to entry within the class constructor itself. You can override this behaviour by specifying a different member constructor (usually a copy constructor) in the initialization list and avoid having to do a separate assignment operation inside the constructor body.

```
class CtorInit
{
public:

    // no list
    CtorInit(const MyClass& t) // m_Value default ctor called here
    automatically
    {
        m_Value = t; // m_Value assignment operator called
    }

    // with list
    CtorInit(const MyClass& t) : m_Value(t) { } // m_Value copy ctor
    called
};
```

In addition, the following scenarios nonetheless only work with member initializer list:

1. Initialization of non-static const data members;
2. Initialization of reference members;
3. Initialization of member objects which do not have default constructor;
4. Initialization of base class through parameterized constructor;

Lastly, place the member variables in the initializer list in the same order as they are declared within the class.

- A public member function must never return a non-const reference or pointer to member data.

By allowing a user direct access to the private member data of an object, this data may be changed in ways not intended by the class designer

```
class Account
{
public:
    Account( int myMoney ) : moneyAmount( myMoney ) {};
    const int& getSafeMoney() const { return moneyAmount; }

    int& getRiskyMoney() const { return moneyAmount; } // No!
    // ...

private:
    int moneyAmount;
};
```

- Do not define implicit conversions. Use the *explicit* keyword for single-argument constructors, and multi-argument constructors with all but the first argument given a default value

```
class Foo
{
    explicit Foo(int x, double y = 1.0);
    explicit Foo(int width);
    ...
};
```

- Use *multiple inheritances* with care. Try to avoid situations where *virtual inheritance* has to be deployed as there is generally a way to rewrite the code to avoid it.

Functions

- Avoid long and complex functions
- Avoid functions with too many arguments.

If a function has to take a lot of arguments it normally can be replaced by a class object.

- In the function arguments use constant reference, or reference-to-constant, instead of pass-by-value. This will avoid the *object slicing* problem.

The exception of this rule is if the data type is a native data type or a pointer.

```
void fool( MyClass s );  
MyClass a;  
fool( a ); // call-by-value. Danger! Object slicing can happen here!
```

- The names of arguments must be consistently specified at both function declarations and function definitions, even though the compiler will pass with function declarations with no argument names.

Parameter names may be omitted only if the parameter is unused and its purpose is obvious.

- Macros created through the *#define* directive is not strictly prohibited but must be used with great caution and programmers must understand exactly how macro function behaves:

```
#define SQUARE(X) X * X

int main(int argc, char* argv[])
{
    int y = 5;

    printf("%d\n", SQUARE(++y));
    return 0;
}

// Caution: SQUARE(++y) expands into ++y*++y and returns a value of
// 49, instead of the intended 36.
```

Namespaces

- With few exceptions, place code in a namespace. Namespaces should have unique names based on their roles in the project.
- Unnamed namespaces are allowed in .cpp files, to avoid link time naming conflicts. Do not however use unnamed namespaces in .h files.

```
namespace    // This is in a .cpp file.
{
    // The content of a namespace is not indented.
    //
    // This function is guaranteed not to generate a colliding
    symbol
    // with other symbols at link time, and is only visible to
    // callers in this .cc file.
    bool UpdateInternals(Frobber* f, int newval)
    {
        ...
    }
} // namespace
```

- Try to classify names with namespace names in all places, such as `MyNamespace::myVar`. Do not use a *using directive* on a namespace to make all names from a namespace available. It's however ok to use *using* on a particular name.

```
using MyNamesapce::myNum;    // This is OK
int _tmain(int argc, _TCHAR* argv[])
{
    std::cout << myNum << std::endl;
    return 0;
}
```

```
Using namespace MyNamesapce; // This is NOT OK!
int _tmain(int argc, _TCHAR* argv[])
{
    std::cout << myNum << std::endl;
    return 0;
}
```

Lambda Expressions

- Never write lambda expressions all in one line

```
// Good
sort(v.begin(), v.end(), [](int x, int y)
{
    return Weight(x) < Weight(y);
});
```

```
// Not possible to put breakpoint to the key line in the lambda body
sort(v.begin(), v.end(), [](int x, int y) { return Weight(x) <
Weight(y); });
```

- Use default captures judiciously to aid readability. However when capturing the `this` pointer or if the lambda will escape the current scope, always capture by reference explicitly

```
{
    Foo foo;
    ...
    executor->Schedule([&] { Frobnicate(foo); })
    ...
}
// BAD! The fact that the lambda makes use of a reference to `foo` and
// possibly `this` (if `Frobnicate` is a member function) may not be
// apparent on a cursory inspection. If the lambda is invoked after
// the function returns, that would be bad, because both `foo`
// and the enclosing object could have been destroyed.
```

C++ Coding Standards: 101 Rules, Guidelines, and Best Practices
By Herb Sutter, Andrei Alexandrescu

C++ Coding Standards
By Joseph Ye (Mizuho Internal Document)

Google Style Guide
<https://google.github.io/styleguide/cppguide.html>

The End